

Coin Hunter Bot — Architecture & Deployment Guide

ربات شکارچی کوین — راهنمای معماری و استقرار

A multi-tier autonomous agent for hunting newly-emerged CPU-mineable cryptocurrencies, designed for an operator with structural electricity advantage and a long-term accumulation thesis.

طراحی شده برای اپراتوری با CPU، ربات چندلایه خودکار برای شکار ارزهای دیجیتال نوظهور قابل ماینینگ با مزیت ساختاری برق و تز انباشت بلندمدت.

Mission Snapshot

- Hunt CPU-mineable coins launched in the last 90 days, market cap \$50k–\$50M
- Survival-first analysis (not price prediction)
- 24/7 autonomous operation with weekly self-improvement
- Bilingual outputs (English + Persian)
- Privacy-respecting hybrid model strategy
- Encrypted storage throughout

File Manifest

File	Tier	Purpose	Mutability
CORE_PRINCIPLES.md	All	Mission, edge, hard filters, never-do list	IMMUTABLE (Armin-only)
orchestrator_prompt.md	Tier 3	Main synthesis prompt for Claude Opus	Protected (structure immutable, content guarded)
tier1_scout_prompt.md	Tier 1	Fast triage prompt for local LLM	Mutable via Tier 4 with approval

tier2_forensics_prompt.md	Tier 2	Evidence gathering for local LLM	Mutable via Tier 4 with approval
tier4_meta_prompt.md	Tier 4	Weekly self-improvement layer	Protected
output_schema.json	Tier 3	Strict JSON schema for verdicts	IMMUTABLE (downstream tools depend)
tactics.yaml	All	Heuristic weights and thresholds	Mutable via Tier 4 with approval

Infrastructure Architecture

HETZNER CCX VPS (primary, 24/7)

- Ubuntu 24 LTS, LUKS full-disk encryption
- Docker + docker-compose
- Ollama with Qwen 2.5 14B (Tier 1 + Tier 2)
- Python orchestrator (Tier 3 caller via Anthropic SDK)
- MinIO local for warm storage (S3-compatible)
- SQLite + SQLCipher for hot state
- cron jobs for tier schedules
- WireGuard VPN for operator SSH

↓ encrypted offsite backup

BACKBLAZE B2 / WASABI

- rclone with crypt remote
- Keys stored OFFLINE on operator hardware
- Daily incremental backups of state
- Weekly full backups

↓ alerts

OPERATOR NOTIFICATIONS

- Signal/Telegram for ACCUMULATE verdicts
- Email for weekly Tier 4 reports
- Push for drift alerts (HIGH severity)

Operational Cadence

Tier	Frequency	Model	Cost Class
Tier 1: Scout	Every 1 hour	Local LLM (Qwen 2.5)	Free
Tier 2: Forensics	Every 6 hours (on Tier 1 survivors)	Local LLM (Qwen 2.5)	Free
Tier 3: Synthesis	Daily (on Tier 2 survivors)	Claude Opus	~\$1-3/day
Tier 4: Meta-review	Weekly (Sundays 02:00 UTC)	Claude Sonnet	~\$5/week

Estimated total LLM cost: **\$50-100/month**. Infrastructure cost: **~\$30-50/month** (Hetzner + storage). **Total operating cost: ~\$80-150/month**.

Security Model

Storage Encryption Layers

1. **Hot data** (state, cache, working sets):
 - SQLite + SQLCipher
 - Encryption keys derived from operator-provided password at startup
 - Password never persisted to disk
2. **Warm data** (chat logs, analysis outputs, dossiers):
 - MinIO with client-side encryption via `age`
 - Age recipients stored in operator's password manager
 - VPS only has public keys; cannot decrypt without operator action
3. **Cold data** (backups, raw scraped data):
 - rclone crypt remote → Backblaze B2 or Wasabi

- Encryption keys on offline hardware (Yubikey-protected if possible)
- Provider sees only encrypted blobs

Key Management Principle

No key required to decrypt warm or cold data is ever stored on the VPS. If the VPS is compromised, attacker gets ciphertext only.

The bot CAN write encrypted data (using public keys / asymmetric mode) but cannot decrypt its own historical archives without operator action. This intentionally limits self-modification scope.

Hybrid Model Strategy Rationale

Tier	Model Choice	Reasoning
Tier 1	Local Qwen 2.5 7B	High volume, simple decisions, public data — no API cost, no privacy concern
Tier 2	Local Qwen 2.5 14B	Investigation work, structured output, moderate complexity. Local because dossiers might contain hypotheses about specific addresses/wallets — privacy matters here.
Tier 3	Claude Opus	Nuanced reasoning, counter-narrative generation, calibration. Worth the cost for verdict-grade synthesis.
Tier 4	Claude Sonnet	Weekly meta-review. Sonnet's instruction-following is sufficient; Opus is overkill.

Self-Improvement Mechanism

What Tier 4 CAN modify (with operator approval flag):

- Content within sections of `tier1_scout_prompt.md`
- Content within sections of `tier2_forensics_prompt.md`
- Heuristic weights in `tactics.yaml`

- Threshold values (within Hard Filter constraints)

What Tier 4 CANNOT modify:

- `CORE_PRINCIPLES.md`
- 9-section structure of orchestrator
- `output_schema.json`
- Core identity statements

Approval Flow

1. Tier 4 proposes change → writes to `pending_changes/PROPOSAL-ID/`
2. Proposal includes: hypothesis, expected impact, rollback condition, `review_date`
3. Each proposal git-committed in proposal branch
4. Operator reviews weekly digest
5. Operator sets `auto_apply: true` in `tactics.yaml` for proposals to take effect
6. After application, performance monitored against rollback condition
7. Automatic revert if rollback condition triggers

Drift Detection

Tier 4 explicitly checks every week for:

- Mission creep (analyzing out-of-scope coins)
- Hype contamination (less skeptical language)
- Confirmation bias (over-recommending)
- Edge neglect (forgetting electricity advantage)
- Adversarial erosion (not generating counter-narratives)

HIGH severity drift alerts pause new recommendations until operator reviews.



Data Sources

Primary (real-time polling):

- CoinGecko API:
 - `/coins/list/new` (every 30 min) — latest listings
 - `/coins/markets` filtered by market cap (every 1 hour)
 - Category filters: `cpu-mineable`, `proof-of-work`
- SRBMiner changelog (every 6 hours) — diff for new algorithm additions
- XMRig + cpuminer-opt GitHub releases (every 12 hours)

Secondary (forensics):

- GitHub API (commits, contributors, age, license)
- Coin-specific block explorers
- DexScreener / GeckoTerminal for DEX data
- BitcoinTalk via scraper
- Discord/Telegram via official APIs where keys available
- Twitter/X via Nitter mirrors (privacy-preserving)

Algorithmic intelligence:

- SRBMiner adds new CPU algorithms regularly (e.g., `randomalpha`, `randomjuno`, `yespowereqpay`) — diff this changelog to discover genuinely-emerging CPU coins before market notices



Deployment Sequence

Phase 1: Foundation (Week 1-2)

1. Provision Hetzner CCX VPS (8 vCPU, 32GB RAM recommended for 14B model)
2. Harden: `ufw`, `fail2ban`, SSH key-only, WireGuard
3. Install Ollama + pull Qwen 2.5 14B (Q4 quantization)
4. Setup MinIO + age + rclone with offline-key model
5. Setup SQLCipher database with schema
6. Deploy CORE_PRINCIPLES.md (hash and commit to git)

Phase 2: Tier 1+2 (Week 2-3)

1. Deploy local LLM with tier 1+2 prompts
2. Build CoinGecko / GitHub / explorer adapters
3. Build SRBMiner changelog watcher
4. Run dry mode for 1 week — observe outputs, do NOT call Tier 3
5. Manual review of Tier 2 dossiers for quality

Phase 3: Tier 3 (Week 4)

1. Add Anthropic API key (with budget caps)
2. Setup `max_budget_usd` parameter (\$50/day initial cap)
3. Run Tier 3 on accumulated Tier 2 dossiers
4. Manual review of all Tier 3 verdicts for first 2 weeks
5. Refine prompts based on observed misalignments

Phase 4: Tier 4 + Self-Improvement (Week 5+)

1. Enable Tier 4 weekly reviews
2. ALL proposed changes set to `auto_apply: false` initially
3. Operator manually approves each change for first month
4. After confidence established, can enable selective auto-apply for low-risk change types

Phase 5: Alerting & Polish (Week 6+)

1. Setup Signal/Telegram notifications
2. Weekly email digest of Tier 4 report
3. Drift alert pipeline
4. Performance dashboard (precision, recall, recommendations/month)

⚠ Known Risks & Mitigations

Risk	Mitigation

Local LLM quality degradation on edge cases	Tier 3 (Claude) catches; Tier 2 includes confidence levels
Goal drift in self-improvement	CORE_PRINCIPLES immutable; Tier 4 drift detection; automatic rollback
API key compromise	Keys in operator-only encrypted vault; rotate quarterly; budget caps
VPS compromise	Encrypted storage; backup keys offline; minimal blast radius
CoinGecko API rate limits	Multi-tier caching; respect 30 calls/min on free tier; consider paid plan
Hallucinated coin data	Tier 2 requires source for every claim; Tier 3 marks LOW confidence on unsourced
Operator over-trust in verdicts	Bot NEVER executes trades; verdicts are inputs to operator decisions only
Survivorship bias in comparable coins	Tier 3 explicitly told to look at FAILED comparables, not just successful ones



Success Criteria (6-month evaluation)

The system is successful if at 6-month mark:

- **Precision (STRONG ACCUMULATE):** $\geq 60\%$ of recommended coins still alive and trading
- **False positive rate:** $< 25\%$ of recommendations dead within 90 days
- **Recommendation cadence:** 5-10 per month (combined STRONG + CAUTIOUS)
- **Edge capture:** $\geq 30\%$ of recommendations are in "edge zone" (operator-profitable but not average-profitable)
- **Drift incidents:** 0 HIGH severity, < 3 MEDIUM severity
- **Operator burden:** < 4 hours/week of review time



Operator Authority (Reminder)

The bot is a tool. Armin is in command. The bot:

- Never trades
- Never moves funds
- Never connects to mining hardware (active or parked)
- Always explains reasoning
- Always flags uncertainty
- Always defers to operator instructions

If you (Armin) at any time want to pause, modify, or shut down the bot — that is an operator command, not a violation of the bot's principles. The bot's principles guide its autonomous behavior, not its responsiveness to you.



Next Steps

1. Review all prompt files
2. Adjust threshold values in `CORE_PRINCIPLES.md` if needed
3. Choose VPS provider and provision
4. Begin Phase 1 deployment

For prompt refinement, the natural next iteration is to:

- Build `tactics.yaml` with initial threshold values
 - Build minimal Python harness that orchestrates the tiers
 - Build test fixtures (sample coin data) to dry-run prompts
-

Repository structure suggested:

```
coin_hunter_bot/  
├─ CORE_PRINCIPLES.md          (immutable)  
├─ orchestrator_prompt.md  
├─ tier1_scout_prompt.md  
├─ tier2_forensics_prompt.md  
├─ tier4_meta_prompt.md  
└─ output_schema.json
```

```
|─ tactics.yaml                (to be built)
|─ README.md                  (this file)
|─ harness/
|   |─ orchestrator.py
|   |─ tier1_runner.py
|   |─ tier2_runner.py
|   |─ tier3_runner.py
|   └─ tier4_runner.py
|─ adapters/
|   |─ coingecko.py
|   |─ github.py
|   |─ explorers/
|   └─ srbminer_changelog.py
|─ storage/
|   |─ sqlcipher_state.py
|   └─ minio_warm.py
|─ tests/
|   └─ fixtures/
└─ pending_changes/           (Tier 4 proposals awaiting approval)
```

END OF README